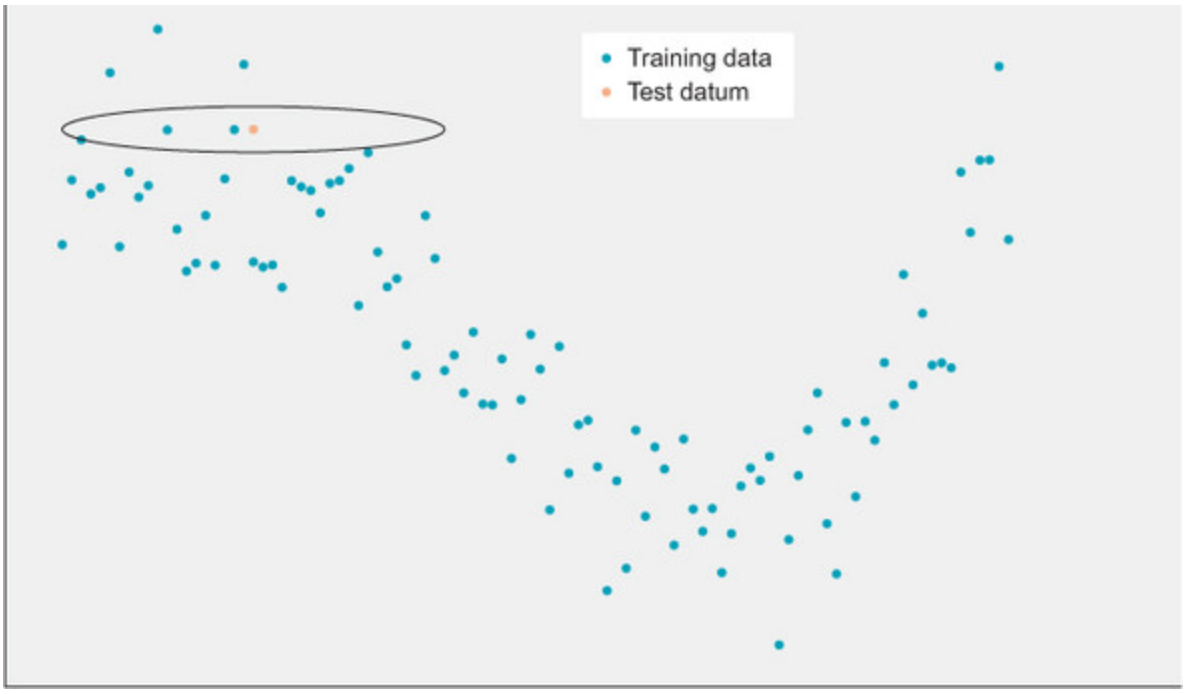


In this case, however, we have to choose how many points qualify as being the nearest neighbors to the input x, and it is problematic since all of these nearest neighbors contribute equally to the outcome. Ideally, we could weight (or attend to) all the points in the data set according to how similar they are to the input, and then take the weighted sum of their corresponding y_i to make a prediction. We'd need some function, $f{:}X \rightarrow A$: a function that takes an input $x \in X$ and returns a set of attention weights $a \in A$ that we could use to perform this weighted sum. This procedure is essentially exactly what we'll do in attention models, except that the difficulty lies in deciding how to efficiently compute the attention weights.

In general, a *self-attention model* seeks to take a collection of objects and learn how each of those objects is related to the other objects via attention weights. In graph theory, a graph is a data structure, $G = (N,E)$, i.e., a collection of nodes, N, and edges (connections or relations) between nodes, E. The collection, N, might just be a set of node labels such as $\{0,1,2,3,...\}$, or each node might contain data, and thus each node might be represented by some feature vector. In the latter case, we can store our collection of nodes as a matrix $N{:}\mathbb{R}^{n \times f}$, where f is the feature dimension, such that each row is a feature vector for that node.